

Licenciatura Engenharia Informática e Multimédia
Instituto Superior de Engenharia de Lisboa
Ano letivo 2024/2025

Interação em Ambientes Virtuais

Trabalho Prático Nº1

Turma: 61D

Nome: João Ramos Número: 50730

Nome: Fábio Pestana Número: 50756

Professor: Diogo Cabral

Data: 21 de Março de 2025

Índice

Lista de Figuras 3
Introdução 4
O que é *Voxel*? 5
Blocos, *Texture Atlas* e *Chunks* 5
Classe Block e *texture atlas* 5
O que são *chunks*? 7
Classe Chunk 8
Geração do mundo 8
Classe Utils 9
Classe World 10
Interação do jogador com o mundo 11
Classe BlockInteraction 11
Classe HandAndBuildBlock 13
Conclusão 15
Bibliografia 16

Lista de Figuras

Figura 1 - Exemplo de mundo Minecraft 5
Figura 2 - *Texture Atlas* utilizado 6
Figura 3 - Desenho explicativo da lógica das texturas 7
Figura 4 - Exemplo de um *chunk* no Minecraft 7
Figura 5 - Teste com terrenos menos suaves (smooth = 0.005f) 8
Figura 6 - Demonstração dos minérios numa gruta 10
Figura 7 - *Hotbar* utilizada no projeto 13
Figura 8 - Demonstração da troca do bloco selecionado 13
Figura 9 - Exemplo de construção por parte do jogador 14

Introdução

Este trabalho tem como objetivo o desenvolvimento de um ambiente tipo Minecraft utilizando o motor de jogo Unity. Os alunos tiveram como base o código desenvolvido nas aulas, com o intuito de criar um mundo virtual composto por blocos (*voxels*) que podem ser interagidos pelo jogador. O projeto foi dividido em várias etapas, desde a criação de blocos individuais até a geração procedural de um mundo vasto e interativo. Além disso, foram utilizadas técnicas de otimização, como o uso de *texture atlas* e a combinação de malhas (*meshes*), para garantir que o jogo fosse executado de forma eficiente, mesmo com um grande número de blocos.

O que é *Voxel*?

Este projeto da unidade curricular Interação em Ambientes Virtuais têm como base a utilização de *Voxels* (*Volume Element*). Estes são uma representação tridimensional de um pixel, ou seja, representam um valor de uma grelha num espaço 3D. Como exemplo principal de um jogo *Voxel*, temos o jogo Minecraft em que este trabalho se baseia.



Figura 1 - Exemplo de mundo Minecraft

Visto que o ambiente a ser desenvolvido é representado em *Voxels*, o mundo criado será composto por diversos blocos, alguns diferentes de outros. Para a implementação, primeiramente, criámos uma classe `Block` que representa um dos inúmeros blocos que compõe o ambiente.

Blocos, *Texture Atlas* e *Chunks*

Classe Block e *texture atlas*

Primeiramente definimos os seis lados do cubo e os diferentes tipos de blocos (Grass, Dirt, Stone, Diamond, Coal, Iron, Cobblestone, Tnt, Air). Cada tipo de bloco tem texturas associadas a si. Para conter as diversas texturas utilizadas nos blocos temos um *texture atlas* que é uma imagem que contem em si múltiplas pequenas imagens, cada uma destas sendo uma textura de um lado de um bloco. Utiliza-se esta ferramenta para se tornar mais eficiente o armazenamento de texturas utilizadas no jogo, reduzindo o alocamento de memória e sobrecarga de *input/output processing system*. O grupo decidiu não utilizar o atlas fornecido pelos docentes da disciplina. De seguida encontra-se o *textures atlas* utilizado.

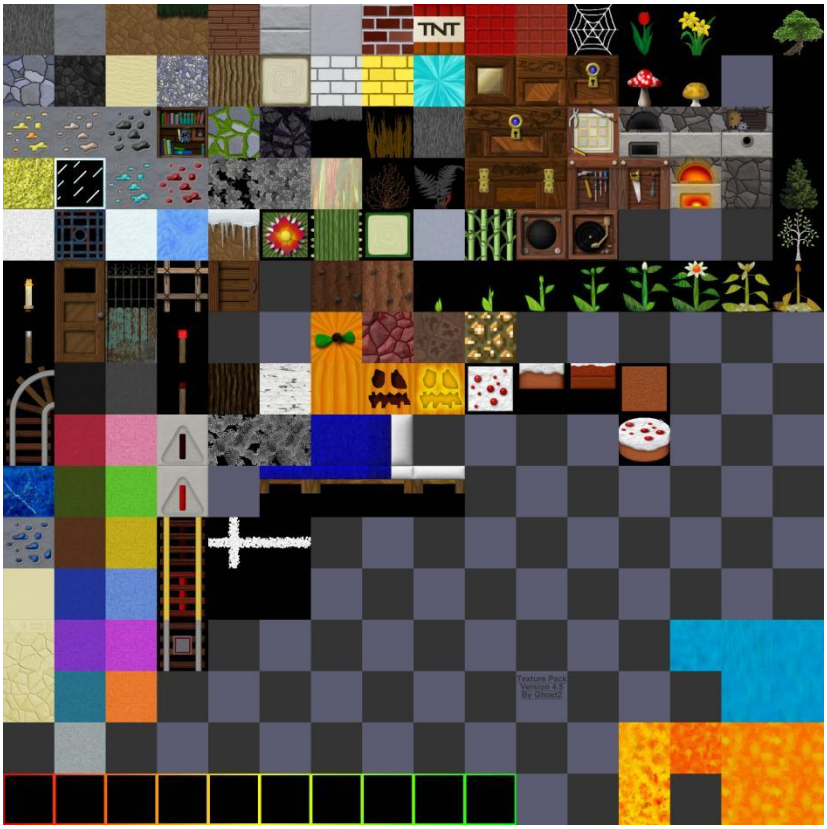


Figura 2 - *Texture Atlas* utilizado

Os atributos da classe `Block` são:

- **Material**: Refere-se ao material do bloco;
- **bType**: Qual é o tipo do bloco;
- **owner**: Refere-se a que *chunk* o bloco pertence (o conceito de chunk será abordado posteriormente);
- **pos**: Posição em que o bloco se encontra referente ao seu *chunk*;
- **isSolid**: Indica se o bloco é sólido, dada a existência de blocos do tipo Air que não o são;
- **blockUVs**: Matriz que armazena as coordenadas das texturas do bloco, onde para cada tipo de bloco utiliza-se uma variável (tipo `Vector2`) que indica as coordenadas do canto inferior esquerdo do bloco no *texture atlas*. Com esta coordenada é possível obter as coordenadas dos restantes cantos da textura. Na seguinte figura, que representa uma textura de um lado de um bloco, para calcular as restantes coordenadas soma-se um `Vector2`, divide-se este vetor por 16 (pois o *texture atlas* escolhido é composto por 16 x 16 texturas), logo com esta divisão e com a soma do vetor obtêm-se as texturas desejadas.

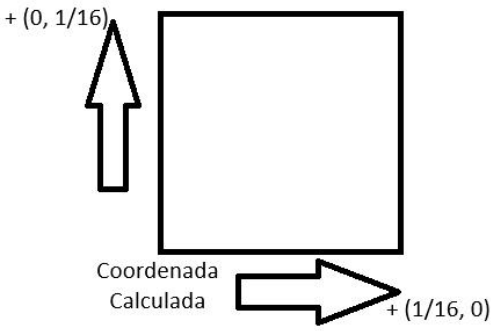


Figura 3 - Desenho explicativo da lógica das texturas

Temos como métodos notáveis:

- **`setType(BlockType blockType)`**: Define o tipo do bloco e se ele é solido (`isSolid`);
- **`CreateQuad(Cubeside side)`**: Cria um quadrado que representa um das faces do bloco;
- **`ConvertToLocalIndex(int i)`**: Converte o indice relativo a outra *chunk* num indice referente à *chunk* do bloco.
- **`HasSolidNeighbour(int x, int y, int z)`**: Verifica se existe algum bloco sólido vizinho na posição dada;
- **`Draw()`**: Desenha as faces do bloco caso não exista nenhum vizinho sólido no lado da face. Isto otimiza a renderização visto que apenas as faces visíveis serão renderizadas.

O que são *chunks*?

Dado que a geração e renderização de milhares de blocos seria um processo extremamente pesado, o sistema de *chunks* (16x16x16 blocos) é implementado de modo a otimizar este processo, visto que os *chunks* permitem agrupar blocos numa única malha, o que reduz chamadas do *draw* (processo de renderização). Para a implementação deste sistema de *chunks* criou-se a classe `Chunk` que representa uma parte do mundo.



Figura 4 - Exemplo de um *chunk* no Minecraft

Classe Chunk

Primeiramente definimos:

- **chunkdata**: Matriz que armazena os blocos dentro do *chunk*;
- **goChunk**: É o `GameObject` que representa o *chunk*;
- **ChunkStatus**: Enumerator com os estados do *chunk* (DRAW e DONE);
- **status**: Representa o estado atual do *chunk*;
- **material**: Material usado para os blocos;
- **probDiamond**: Probabilidade do bloco ser do tipo diamante;
- **probIron**: Probabilidade do bloco ser do tipo ferro;
- **probCoal**: Probabilidade do bloco ser do tipo carvão.

E temos como métodos notáveis:

- **`BuildChunk()`**: Gera a estrutura dos *chunks* e define os blocos que fazem parte deles. Utiliza outros métodos auxiliares para verificar se devem ser gerados minérios (abaixo da camada de pedra e cada um com uma probabilidade própria) ou blocos com erva, no caso dos blocos estarem à superficie se não estes vão ser do tipo terra (Dirt). Todos os blocos acima do solo são blocos do tipo AIR;
- **`DrawChunk()`**: Renderiza as faces visíveis dos blocos pertencentes ao *chunk*;
- **`CombineQuads()`**: Junta todas as faces individuais dos blocos numa única malha, o que otimiza a renderização.

Geração do mundo

Para gerar o terreno, implementou-se a classe `Utils` que tem como objetivo, através da utilização do *Perlin Noise*, gerar alturas e variações naturais no terreno de forma procedural. Esta abordagem permite criar um terreno que parece orgânico, com colinas, montanhas e outras características.



Figura 5 - Teste com terrenos menos suaves (smooth = 0.005f)

Classe Utils

Primeiramente definimos:

- **smooth**: Serve como suavidade do terreno (quanto menor, mais plano o terreno será);
- **smooth3D**: Uma versão ampliada de *smooth* (10 vezes maior) para ser usada em cálculos de ruído 3D;
- **maxHeight**: Define a altura máxima do terreno. No código, este valor é definido como 150, o que significa que o terreno pode atingir até 150 unidades de altura;
- **octaves**: Define o número de camadas de ruído que serão combinadas para criar o terreno. Cada camada (*octave*) adiciona detalhes ao terreno;
- **persistence**: Controla como a amplitude do ruído diminui a cada camada (*octave*). Um valor de `0.7f` significa que cada camada subsequente terá 70% da amplitude da camada anterior;
- **offset**: É o desvio fixo que evita padrões repetidos do *Perlin Noise*.

E temos como métodos notáveis:

- **`GenerateHeight(float x, float z)`**: Calcula a altura do terreno para um ponto específico utilizando o *Perlin Noise*, o valor do ruído é obtido através da função auxiliar fBM e depois é normalizado para o intervalo [0, maxHeight], através da função Map;
- **`GenerateStoneHeight(float x, float z)`**: Calcula a altura para a camada em que os blocos do tipo STONE começam a aparecer. Utiliza um ruído diferente, com uma frequência maior (3 * `smooth`) e uma persistência ligeiramente aumentada (1.2f * `persistence`), para criar uma camada de pedra mais irregular e profunda;
- **`Map(float newmin, float newmax, float orimin, float orimax, float val)`**: Normaliza um valor (`val`) de um intervalo de `orimin` a `orimax` para um intervalo de `newmin` a `newmax` utilizando `Mathf.Lerp`;
- **`fBM(float x, float z, int octaves, float persistence)`**: Implementa o *fractal Brownian motion* (fBM), que combina várias camadas do *Perlin Noise* (*octaves*) para criar um ruído mais complexo. Cada camada tem uma frequência e amplitude diferentes, controladas pelos parâmetros `octaves` e `persistence`;
- **`fBM3D(float x, float y, float z, int octaves, float persistence)`**: Como o Unity não tem uma implementação para o *Perlin Noise* 3D, este método tem como objetivo implementá-lo utilizando o método anterior fBM, onde realiza-se seis chamadas da função em diferentes combinações de eixos (xy, yx, xz, zx, yz e zy) e retorna a média dos resultados.



Figura 6 - Demonstração dos minérios numa gruta

Agora, com o sistema de *chunks* a funcionar começámos a implementação da geração do mundo. A classe `World` foi criada para controlar a geração do mundo.

Classe World

Primeiramente definimos:

- **player**: Representa o jogador;
- **material**: Material dos blocos;
- **chunkSize**: Define o tamanho de cada *chunk* (16x16x16);
- **radius**: Raio de *chunks* carregados ao redor do jogador, *chunks* fora do raio são destruídas;
- **chunkDict**: Dicionário que armazena todos os *chunks* carregados no mundo;
- **toRemove**: Lista de *chunks* a serem removidos;
- **lastBuildPos**: Última posição do jogador onde os *chunks* foram gerados;
- **drawing**: Booleano que informa se parte do mundo está a ser desenhado ou não.

E temos como métodos notáveis:

- **`CreateChunkName(Vector3 v)`**: Gera um nome único para cada *chunk* dependendo das suas coordenadas no mundo (x,y,z);
- **`BuildRecursiveWorld(Vector3 chunkPos, int rad)`**: Gera *chunks* e expande o mundo recursivamente até o raio ser atingido;
- **`BuildChunkAt(Vector3 chunkPos)`**: Cria um *chunk* numa posição específica caso não exista no dicionário um *chunk* na posição recebida (`chunkPos`);
- **`RemoveChunks()`**: Remove os *chunks* que ficaram fora do raio do jogador;
- **`DrawChunks()`**: Chama `DrawChunk()` para cada *chunk* do dicionário que precisa de ser renderizado,

inicialmente limpa-se a lista `toRemove` (`toRemove.clear()`) para assim realizar a remoção correta dos *chunks*. Sem esta adição no método ocorriam algumas inconsistências, pois a lista teria entradas antigas o que poderia levar a *chunks* que estão no raio do jogador serem removidos;

- **`Drawing ()`**: Começa a *coroutine* `DrawChunks()`;
- **`Building(Vector3 chunkPos, int rad)`**: Começa a *coroutine* `BuildRecursiveWorld (Vector3 chunkPos, int rad)`;
- **`WhichChunk(Vector3 position)`**: Retorna a posição do *chunk* ao qual uma posição (x, y, z) pertence;
- **`Start()`**: é responsável por inicializar o mundo e preparar o ambiente para o jogador, gerando os *chunks* iniciais e posicionando o jogador corretamente;
- **`Update()`**: é responsável por atualizar o mundo conforme o jogador se move, gerando novos *chunks* e removendo os que estão fora do raio do jogador.

Interação do jogador com o mundo

Com o mundo já pronto, queremos proporcionar ao jogador a possibilidade de interagir com o mundo que o rodeia, para tal são criadas as classes `BlockInteraction` e `HandAndBuildBlock`.

A classe `BlockInteraction` tem como principal objetivo gerenciar a interação do jogador com os blocos que compõe o mundo, o que permite a destruição de blocos existentes e a construção de outros blocos onde o jogador quiser.

Já a classe `HandAndBuildBlock` tem como principal objetivo permitir que o jogador alterne entre os diferentes blocos que pode construir no mundo e, ao serem construídos, sejam renderizados corretamente.

Classe BlockInteraction

Primeiramente definimos:

- **cam**: É a câmara do jogador que é utilizada para lançar `Raycasts` para sabermos onde o jogador está a mirar;
- **InteractionType**: Enumerator que define os dois tipos de interações possíveis (DESTROY e BUILD);
- **interactionTupe**: Guarda a interação atual;
- **quad**: Referência ao script que trata da escolha do bloco que vai ser construído (`HandAndBuildBlock`).

E temos como métodos notáveis:

- **`Update()`**: É um método chamado a cada *frame* que verifica se o jogador clicou para interagir com um bloco. Primeiramente deteta o *input* do jogador e verifica se foi um clique com algum botão do *mouse*. Se foi um clique com botão esquerdo, o tipo de interação (`interactionType`) será dada como DESTROY. Caso seja um clique com botão direito, o tipo de interação (`interactionType`) será do tipo BUILD. Posteriormente utiliza um `Raycast` de modo a identificar o bloco que o jogador selecionou, se a ação for

do tipo DESTROY o bloco selecionado será removido. Caso a ação seja do tipo BUILD, um novo bloco será construído na face em que o `Raycast` atingiu o bloco selecionado. Na construção dos blocos é importante destacar que caso o jogador tente interagir com os blocos da borda do *chunk*, ou seja, o jogador deseja construir no *chunk* vizinho, é necessário ajustar as coordenadas para afetar o *chunk* vizinho e não o atual. Para tal, foram feitas as seguintes verificações, após calcular-se as variáveis `blockX`, `blockY`, `blockZ`:

```
if (blockX < 0 || blockX >= World.chunkSize || blockY < 0 || blockY >=
World.chunkSize ||blockZ < 0 || blockZ >= World.chunkSize)
{
	if (blockX < 0)
	{
		chunkX -= World.chunkSize;
		blockX = World.chunkSize - 1;
	}
	else if (blockX >= World.chunkSize)
	{
		chunkX += World.chunkSize;
		blockX = 0;
	}

	if (blockY < 0)
	{
		chunkY -= World.chunkSize;
		blockY = World.chunkSize - 1;
	}
	else if (blockY >= World.chunkSize)
	{
		chunkY += World.chunkSize;
		blockY = 0;
	}

	if (blockZ < 0)
	{
		chunkZ -= World.chunkSize;
		blockZ = World.chunkSize - 1;
	}
	else if (blockZ >= World.chunkSize)
	{
		chunkZ += World.chunkSize;
		blockZ = 0;
	}

	chunkName = World.CreateChunkName(new Vector3(chunkX, chunkY, chunkZ));
}
```

O objetivo deste código é verificar se as coordenadas do bloco encontram-se dentro dos limites do *chunk* atual, se qualquer uma das coordenadas estiver fora dos limites é necessário ajustar as coordenadas do bloco para referenciar o *chunk* vizinho e é necessário ajustar as coordenadas do *chunk*, pois desejamos afetar o vizinho do *chunk* atual e não o prórprio. De seguida, atualiza-se o `chunkName`, com as coordenadas do *chunk* vizinho.

Por fim atualiza-se os *chunks* que foram afetados pelas interações do jogador.

Classe HandAndBuildBlock

Para esta classe, reutilizou-se uma versão antiga da classe `Block` (`Quad`), para assim construir o bloco na mão do jogador (para assim este conseguir perceber qual bloco tem selecionado para colocar no mundo). Primeiramente definimos:

- **material**: Material do bloco;
- **bType**: Tipo do bloco que está a ser segurado e posteriormente será construído (inicialmente de tipo GRASS);
- **buildLocation**: É um `Transform` que informa onde o bloco será construído (mão do jogador);
- **b**: Instância de um `Block` usada para acessar as texturas no Atlas.

E temos como métodos notáveis:

- **`Quad(Cubeside side)`**: Método que cria uma face do cubo através da crição de um `GameObject` "quad";
- **`CombineQuads()`**: Junta as faces do bloco num único `MeshFilter` para otimizar a renderização;
- **`CreateCube()`**: Chama os métodos acima e define a posição do bloco com base na `buildLocation`, para assim colocar o cubo na mão do jogador;
- **`ChangeBlockType(Block.BlockType newType)`**: Atualiza o `bType` com o novo tipo do bloco;
- **`Update()`**: É um método chamado a cada *frame* que verifica se o jogador clicou em alguma tecla de 1 a 8. Se for o caso, é chamado o método `ChangeBlockType()` que muda o tipo do bloco na mão do jogador para o bloco selecionado na *hotbar* (blocos de 1 a 8).



Figura 7 - *Hotbar* utilizada no projeto

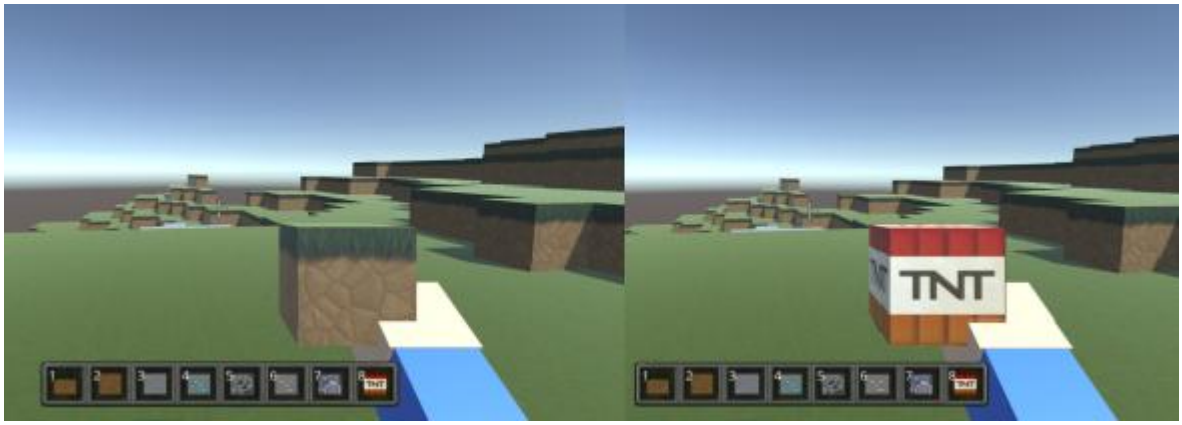


Figura 8 - Demonstração da troca do bloco selecionado

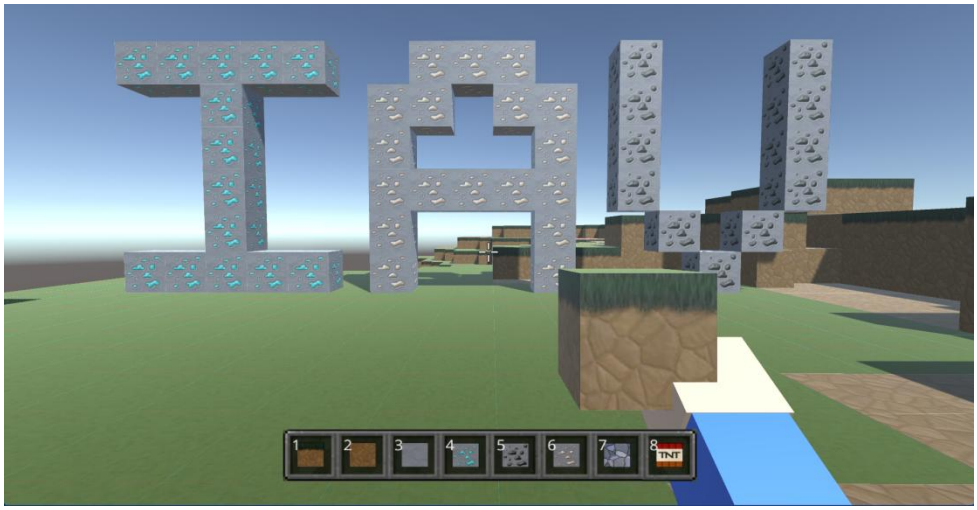


Figura 9 - Exemplo de construção por parte do jogador

Conclusão

O desenvolvimento deste projeto permitiu-nos explorar e aplicar diversos conceitos relacionados com a criação de ambientes virtuais interativos, utilizando o motor de jogo Unity. Através da implementação de um sistema baseado em *voxels*, semelhante ao jogo Minecraft, conseguimos criar um mundo proceduralmente gerado, composto por blocos que podem ser interagidos pelo jogador.

Para futuros trabalhos, seria interessante explorar a adição de mais funcionalidades, como a geração de estruturas pré-definidas (como vilas), e a introdução de entidades que interajam com o ambiente e jogador (animais).

Bibliografia

Consulta:

Vídeos gravados pelo Prof. Arnaldo Abrantes - Moodle

Wikipedia Contributors. (2019, December 9). *Voxel*. Wikipedia; Wikimedia Foundation. https://en.wikipedia.org/wiki/Voxel

Texture atlas. (2022, May 30). Wikipedia. https://en.wikipedia.org/wiki/Texture_atlas